ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



PHÁT TRIỂN ỨNG DỤNG INTERNET OF THINGS (CO3038)

BÁO CÁO BÀI THỰC HÀNH 1

NỀN TẢNG COREIOT VÀ ĐIỀU KHIỂN TỪ XA

Lớp: L01 - Học Kỳ HK261

Ngày nộp: 27/10/2026

GVHD: Thầy Phan Văn Sỹ

Sinh Viên	MSSV
Phạm Công Võ	2313946

TP.Hồ Chí Minh, Tháng 10/2026

THÔNG TIN THỰC HIỆN VÀ ĐÁNH GIÁ KẾT QUẢ

BÀI THỰC HÀNH 1

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả	Chữ ký
1	Phạm Công Võ	2313946	• Hiện thực toàn bộ các bài tập từ Bài 1 đến Bài 5 • Thực nghiệm giả lập Python / Nền tảng CoreIOT Cloud • Soạn thảo tài liệu báo cáo	100%	

Mục lục

1 Tổng Quan và Mục Tiêu Bài Thực Hành 1 **3**
1.1 Giới thiệu . . . 3
1.2 Mục tiêu bài thực hành . . . 3
1.3 Bảng thông số cấu hình thực nghiệm . . . 3

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành **4**
2.1 Bài 1: Tạo thiết bị, thông tin xác thực và nhóm thiết bị (1.5 Điểm) . . . 4
2.1.1 Hình ảnh minh chứng thực nghiệm . . . 4
2.2 Bài 2: Gửi telemetry từ Python kèm cảm biến ánh sáng (2.0 Điểm) . . . 6
2.2.1 Mã nguồn Python gửi Telemetry (Mã 2.1) . . . 6
2.2.2 Hình ảnh minh chứng thực nghiệm . . . 7
2.2.3 Trả lời câu hỏi bài tập 2 . . . 8
2.3 Bài 3: Xây dựng Dashboard giám sát (2.5 Điểm) . . . 9
2.3.1 Hình ảnh minh chứng thực nghiệm . . . 9
2.3.2 Trả lời câu hỏi bài tập 3 . . . 10
2.4 Bài 4: Điều khiển từ xa bằng RPC (2.5 Điểm) . . . 11
2.4.1 Mã nguồn Python xử lý RPC và phản hồi hai chiều (Mã 4.1) . . . 11
2.4.2 Hình ảnh minh chứng thực nghiệm . . . 12
2.4.3 Trả lời câu hỏi bài tập 4 . . . 13
2.5 Bài 5: Chia sẻ Dashboard công khai (1.5 Điểm) . . . 14
2.5.1 Kết quả thực nghiệm & Liên kết truy cập công khai . . . 14
2.5.2 Trả lời câu hỏi bài tập 5 . . . 15

Tài liệu tham khảo **16**

1 Tổng Quan và Mục Tiêu Bài Thực Hành 1

1.1 Giới thiệu

Bài thực hành 1 làm quen với nền tảng điện toán đám mây vạn vật kết nối **CoreIOT** (phát triển trên nền tảng mã nguồn mở ThingsBoard). Hệ thống cung cấp hạ tầng quản lý vòng đời thiết bị, bảo mật danh tính qua mã thông báo bí mật (Access Token), thu thập dữ liệu viễn trắc (Telemetry) và cơ chế điều khiển hai chiều thời gian thực thông qua lời gọi thủ tục từ xa (Remote Procedure Call - RPC) qua giao thức MQTT.

1.2 Mục tiêu bài thực hành

- Đăng nhập và quản trị nền tảng CoreIOT với vai trò Tenant Administrator.
- Khởi tạo thiết bị, cấu hình Access Token bảo mật độc quyền và tổ chức thiết bị vào các nhóm (Entity Groups).
- Viết và chạy chương trình Python sử dụng thư viện MQTT (`paho-mqtt 2.x`) để gửi dữ liệu chuỗi thời gian gồm nhiệt độ, độ ẩm và cường độ ánh sáng lên máy chủ CoreIOT theo chu kỳ 3 giây.
- Thiết kế giao diện Dashboard giám sát thời gian thực với các thẻ chỉ thị số (Value Cards) và biểu đồ chuỗi thời gian (Time series chart).
- Hiện thực cơ chế RPC hai chiều (Two-way RPC) để điều khiển bật/tắt thiết bị chấp hành (LED, Máy bơm và Quạt làm mát) từ Dashboard và phản hồi trạng thái về máy chủ.
- Cấu hình phân quyền kế thừa thực thể và chia sẻ Dashboard công khai (Public Dashboard) xem được trên chế độ ẩn danh không cần đăng nhập.

1.3 Bảng thông số cấu hình thực nghiệm

Bảng 2: Bảng cấu hình thông số kỹ thuật thực nghiệm trên CoreIOT

Thuộc tính	**Giá trị cấu hình**	**Mục đích / Ghi chú**
Họ và tên sinh viên	Phạm Công Võ	Sinh viên thực hiện
Mã số sinh viên	`2313946`	Định danh sinh viên
Tên thiết bị (Device Name)	`Lab1_2313946`	Định danh thiết bị trên CoreIOT
Device Profile	`default`	Luồng xử lý quy tắc mặc định
Credentials Type	`Access token`	Xác thực MQTT qua Username (≥ 12 ký tự)
Nhóm thiết bị (Device Group)	`LAB1_2313946_group`	Gom nhóm quản lý và chia sẻ công khai
Tên Dashboard	`LAB1_2313946 Dashboard`	Bảng điều khiển trực quan hóa dữ liệu
Server Broker	`app.coreiot.io`	Máy chủ đám mây CoreIOT (Port 1883)

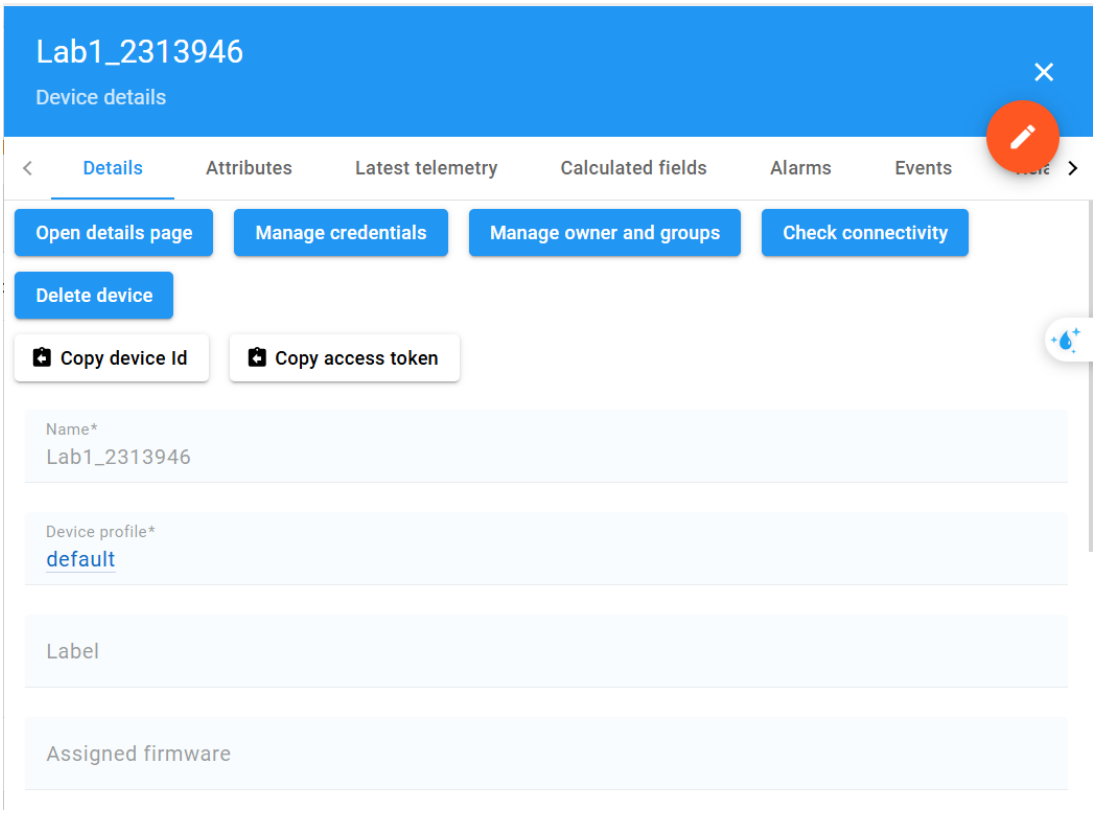
2 Nội Dung Chi Tiết Các Bài Tập Thực Hành

2.1 Bài 1: Tạo thiết bị, thông tin xác thực và nhóm thiết bị (1.5 Điểm)

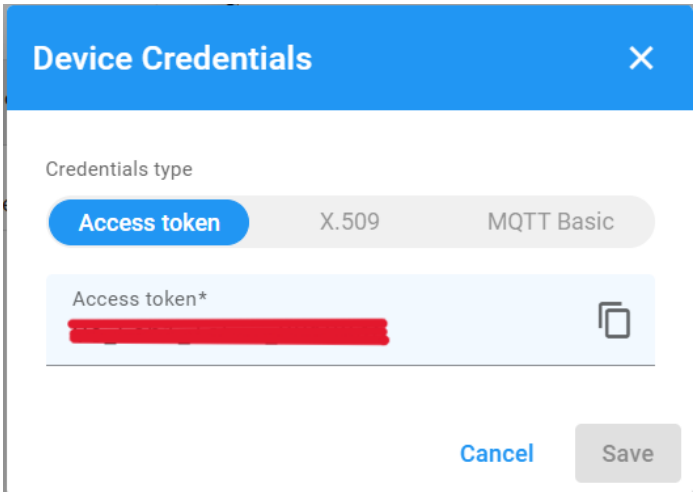
Yêu cầu đề bài (Trích phiếu bài tập)

- **Tạo một thiết bị tên `Lab1_2313946` với Device Profile `default`.**
- **Mở Manage credentials và đặt một Access Token tùy chỉnh do bạn chọn (ít nhất 12 ký tự). Lưu lại.**
- **Tạo một nhóm thiết bị tên `LAB1_2313946_group` và thêm thiết bị của bạn vào nhóm.**
- **Chụp 3 ảnh minh chứng: Trang Details thiết bị, Hộp thoại Credentials (làm mờ token), và Danh sách nhóm.**
- **Trả lời Câu hỏi 1.**

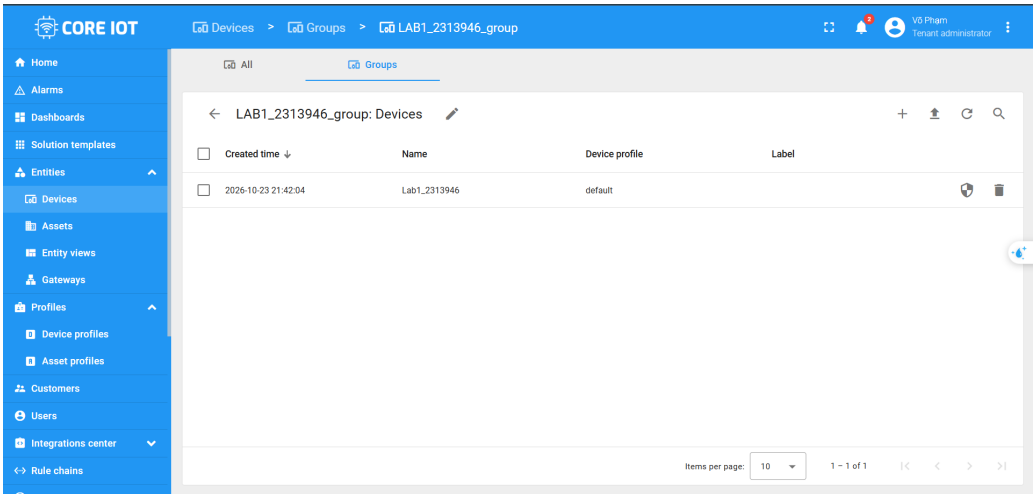
2.1.1 Hình ảnh minh chứng thực nghiệm



Hình 1: Trang chi tiết thiết bị (Tab Details) hiển thị tên thiết bị `Lab1_2313946`



Hình 2: Hộp thoại Device Credentials chọn Access token (giá trị token đã được che bảo mật)



Hình 3: Nhóm thiết bị `LAB1_2313946_group` hiển thị thiết bị bên trong và quyền quản trị viên

Câu hỏi 1: Sự khác nhau giữa hai loại thông tin xác thực Access token và MQTT Basic là gì? Vì sao phải giữ bí mật token?

Trả lời:

- **Sự khác nhau:** *Access token* sử dụng một chuỗi ký tự bí mật duy nhất được truyền trong trường `Username` của bản tin kết nối MQTT (bỏ trống trường Password), giúp cơ chế bắt tay kết nối cực kỳ gọn nhẹ, tối ưu cho các vi điều khiển và thiết bị IoT bị giới hạn tài nguyên tính toán. Trong khi đó, *MQTT Basic* yêu cầu cấu hình đầy đủ bộ ba thông tin gồm: `Client ID`, `User Name` và `Password`.
- **Vì sao phải giữ bí mật token:** Access token đóng vai trò là "chìa khóa định danh độc quyền"của thiết bị trên hệ thống CoreIOT. Nếu token bị rò rỉ, kẻ xấu từ bất kỳ đâu trên Internet đều có thể giả mạo thiết bị để bơm dữ liệu giả mạo (Data Poisoning) làm sai lệch kết quả giám sát, hoặc nguy hiểm hơn là gửi các lệnh điều khiển trái phép can thiệp vào hoạt động của thiết bị phần cứng thực tế.

2.2 Bài 2: Gửi telemetry từ Python kèm cảm biến ánh sáng (2.0 Điểm)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Bắt đầu từ script `lab1_device.py` và đặt Access Token bí mật của bạn.**
- **Thêm khóa telemetry thứ ba tên `light` với giá trị số nguyên ngẫu nhiên từ 0 đến 1000 (`random.randint(0, 1000)`).**
- **Đổi chu kỳ gửi dữ liệu từ 5 giây thành 3 giây (`time.sleep(3)`).**
- **Chạy script ít nhất một phút và kiểm tra dữ liệu trên CoreIOT.**
- **Chụp ảnh màn hình Terminal và tab Latest telemetry của thiết bị.**
- **Trả lời Câu hỏi 2.**

2.2.1 Mã nguồn Python gửi Telemetry (Mã 2.1)

Listing 1: Vòng lặp gửi dữ liệu viễn trắc định kỳ mỗi 3 giây trong lab1_device.py

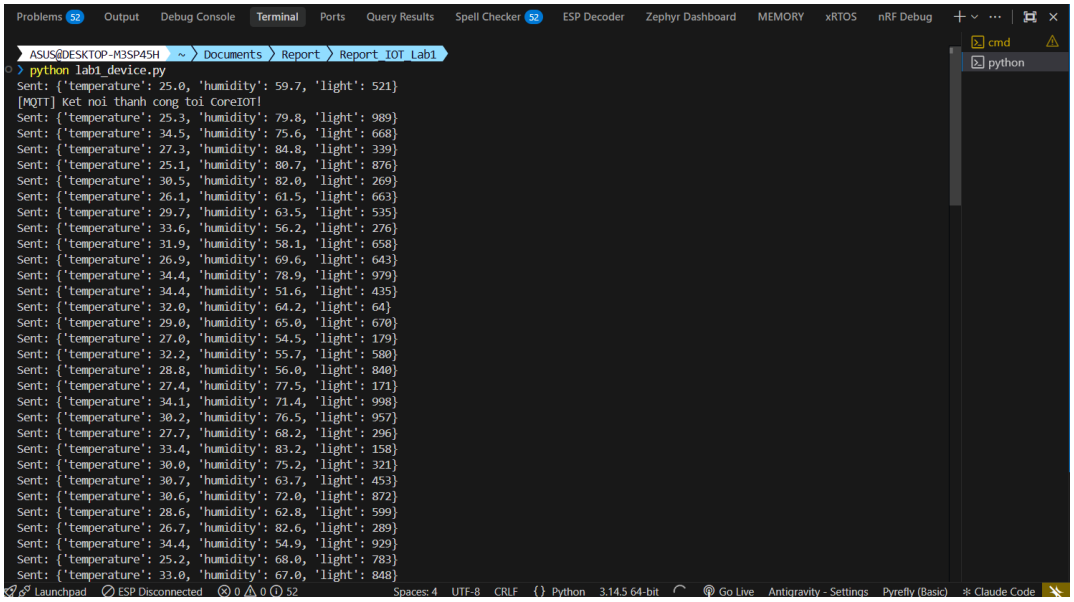
```
try:
    while True:
        # Thu thap cac thong so cam bien mo phong
        temp = round(random.uniform(25.0, 35.0), 1)
        humi = round(random.uniform(50.0, 85.0), 1)
        light = random.randint(0, 1000)

        # Dong goi payload JSON gom du 3 dai luong
        payload = {
            "temperature": temp,
            "humidity": humi,
            "light": light
        }

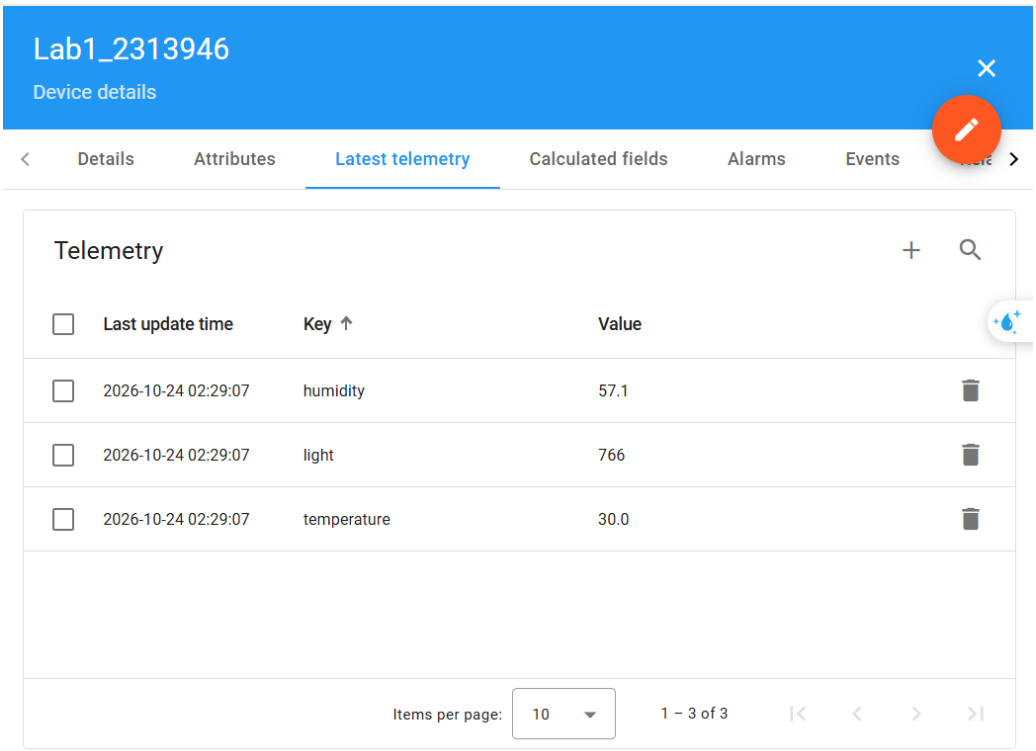
        # Gui du lieu len topic Telemetry chuan cua CoreIOT
        client.publish("v1/devices/me/telemetry", json.dumps(payload), qos=1)
        print(f"Sent: {payload}")

        # Yeu cau de bai: Chu ky truyen gui du lieu la 3 giay
        time.sleep(3)
except KeyboardInterrupt:
    print("\nDang ngat ket noi...")
```

2.2.2 Hình ảnh minh chứng thực nghiệm



Hình 4: Terminal hiển thị kết nối thành công và các dòng Sent chứa cả 3 khóa bao gồm `light`



Hình 5: Tab Latest telemetry của thiết bị `Lab1_2313946` hiển thị đầy đủ 3 khóa dữ liệu

2.2.3 Trả lời câu hỏi bài tập 2

Câu hỏi 2: Script gửi telemetry lên MQTT topic nào, và một bản tin trông như thế nào (viết một ví dụ payload JSON)?

Trả lời:

- **MQTT Topic:** Script gửi dữ liệu viễn trắc lên topic chuẩn theo quy ước ThingsBoard/CoreIOT:

  `v1/devices/me/telemetry`

  Trong đó ký tự `"me"` đại diện cho chính thiết bị sở hữu Access Token đang được kết nối.

- **Ví dụ bản tin JSON Payload thực tế:**

```
{"temperature": 28.5, "humidity": 68.2, "light": 650}
```

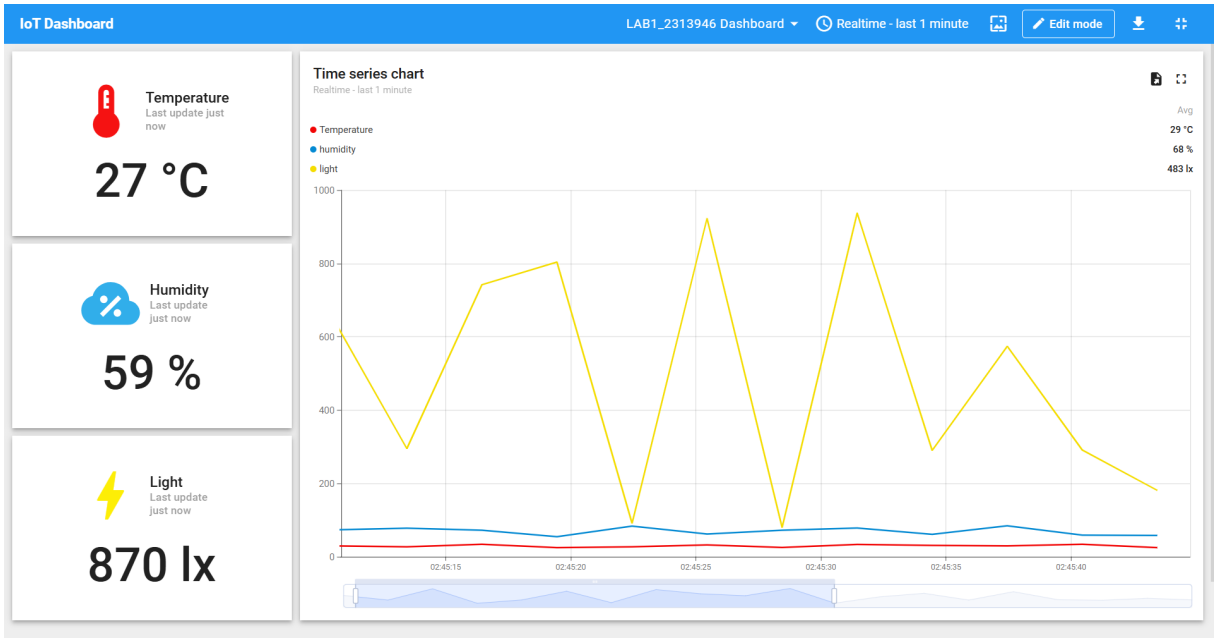
  Bản tin gồm 3 cặp khóa-giá trị: nhiệt độ đo theo độ C, độ ẩm tương đối tính theo %, và cường độ ánh sáng đo theo đơn vị Lux.

2.3 Bài 3: Xây dựng Dashboard giám sát (2.5 Điểm)

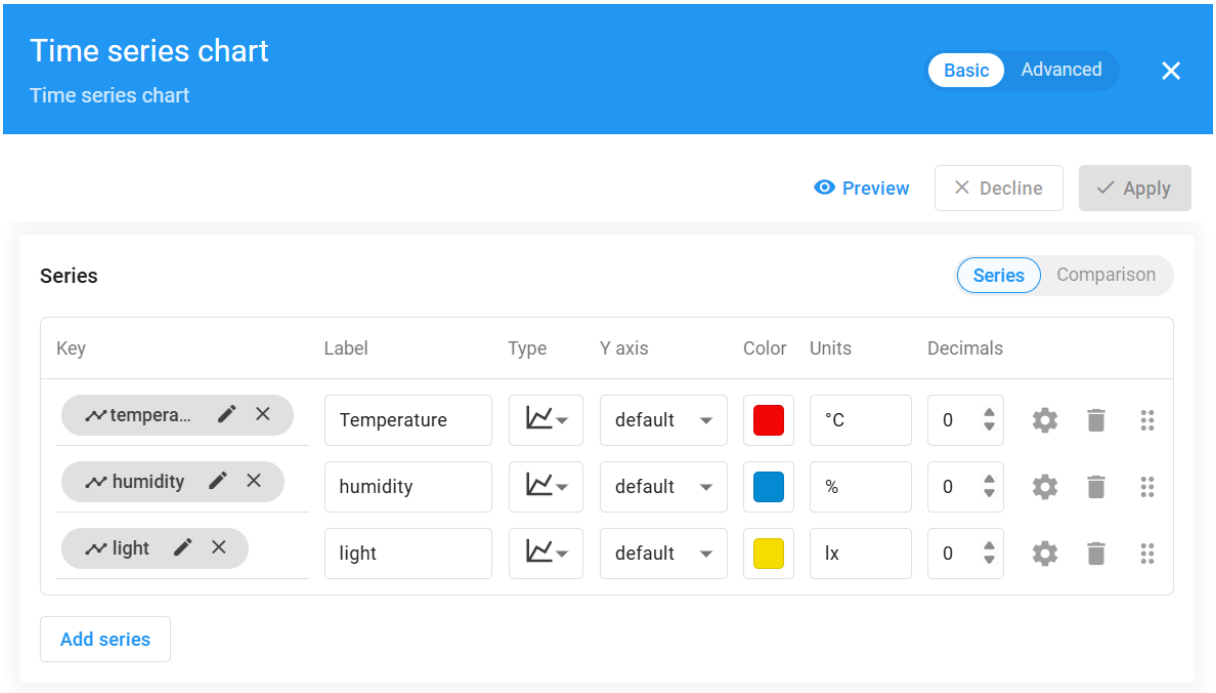
Yêu cầu đề bài (Trích phiếu bài tập)

- **Tạo một Dashboard mới tên `LAB1_2313946 Dashboard`.**
- **Thêm ba widget Value card: Nhiệt độ (°C), Độ ẩm (%), và Ánh sáng (lx), cấu hình nhãn, đơn vị và biểu tượng phù hợp.**
- **Thêm một Widget Time series chart hiển thị đồng thời cả ba khóa dữ liệu dưới dạng các chuỗi (series) riêng biệt.**
- **Sắp xếp gọn gàng và lưu lại Dashboard.**
- **Trả lời Câu hỏi 3.**

2.3.1 Hình ảnh minh chứng thực nghiệm



Hình 6: Toàn bộ giao diện `LAB1_2313946 Dashboard` hiển thị trực quan 3 thẻ số và biểu đồ đường



Hình 7: Cấu hình trình chỉnh sửa widget Time series chart hiển thị đủ 3 chuỗi dữ liệu (Series)

2.3.2 Trả lời câu hỏi bài tập 3

Câu hỏi 3: Khi bạn thêm Value card lần đầu, vì sao khóa light có thể không xuất hiện trong danh sách data key? Bạn khắc phục bằng cách nào?

Trả lời:

- **Nguyên nhân:** Nền tảng CoreIOT chỉ tự động phát hiện, lập chỉ mục và gợi ý danh mục Data Key trong trình thiết kế widget sau khi máy chủ nhận được ít nhất một bản tin telemetry chứa khóa đó từ thiết bị gửi lên. Nếu script Python chưa chạy hoặc chưa gửi gói tin nào có trường `light`, máy chủ sẽ coi như khóa này chưa từng tồn tại trên thiết bị.
- **Cách khắc phục:**
  1. Khởi động script Python chạy trước khoảng vài chu kỳ để gửi dữ liệu có chứa khóa `light` lên hệ thống, sau đó quay lại trang chỉnh sửa widget.
  2. Hoặc có thể tự nhập trực tiếp chuỗi ký tự `"light"` vào ô tìm kiếm Data Key trên giao diện Datasources rồi nhấn phím **Enter** để thiết lập thủ công.

2.4 Bài 4: Điều khiển từ xa bằng RPC (2.5 Điểm)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Thêm các widget Single Switch cho LED và Pump (Máy bơm) vào Dashboard.**
- **Thêm công tắc thứ ba cho Fan (Quạt) sử dụng các phương thức RPC `getStateFAN` và `setStateFAN`.**
- **Cập nhật script Python để lưu trữ trạng thái Quạt trong từ điển `state`, trả lời các yêu cầu `getState` và `setState`, in ra thông báo rõ ràng khi có thay đổi trạng thái (`-> ON` / `-> OFF`).**
- **Bật/tắt mỗi công tắc, tải lại Dashboard và kiểm tra tính nhất quán trạng thái.**
- **Trả lời Câu hỏi 4.**

2.4.1 Mã nguồn Python xử lý RPC và phản hồi hai chiều (Mã 4.1)

```python
state = {"LED": False, "PUMP": False, "FAN": False}

def on_message(client, userdata, msg):
    # Trich xuat request_id tu topic: v1/devices/me/rpc/request/{request_id}
    request_id = msg.topic.split("/")[-1]
    try:
        req = json.loads(msg.payload)
    except json.JSONDecodeError:
        print("[RPC] Payload JSON khong hop le:", msg.payload)
        return

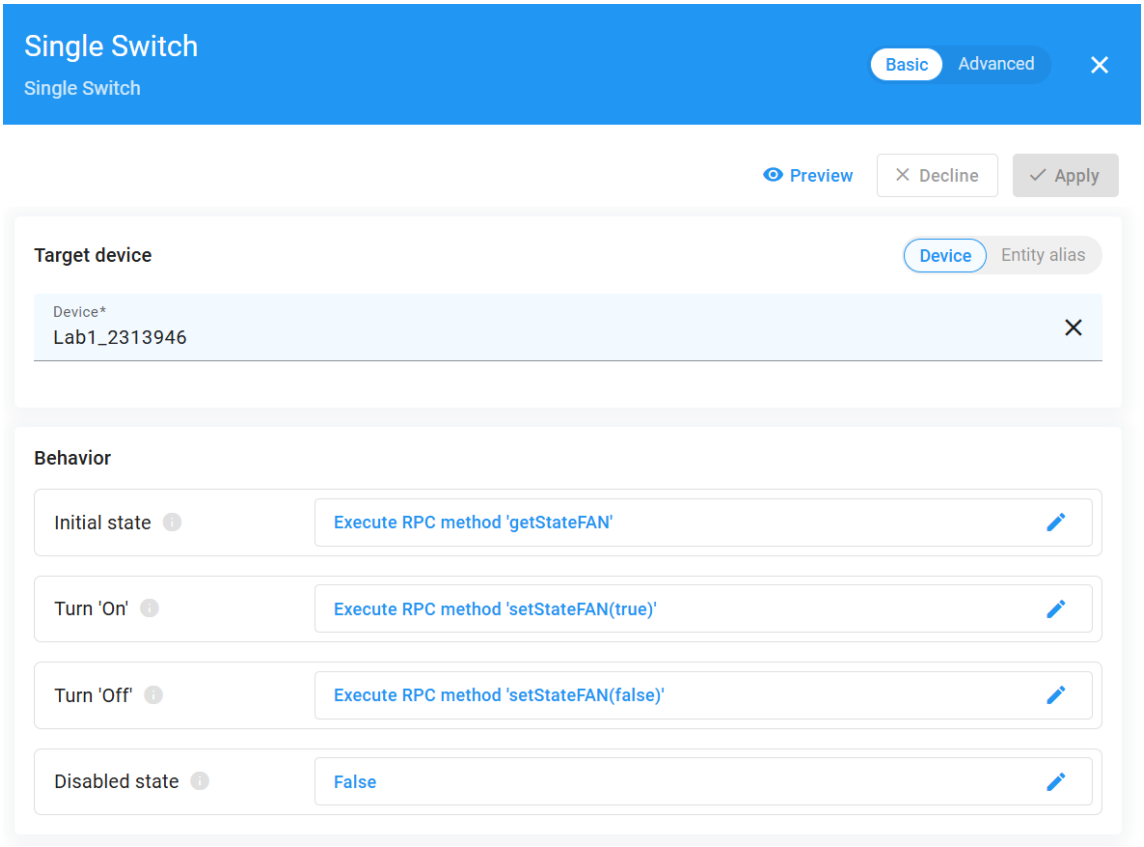
    method = req.get("method", "")
    params = req.get("params")
    result = None

    # Xu ly getState: tra ve trang thai khi widget hoac Dashboard khoi tao
    if method.startswith("getState"):
        name = method[len("getState"):]
        result = state.get(name, False)

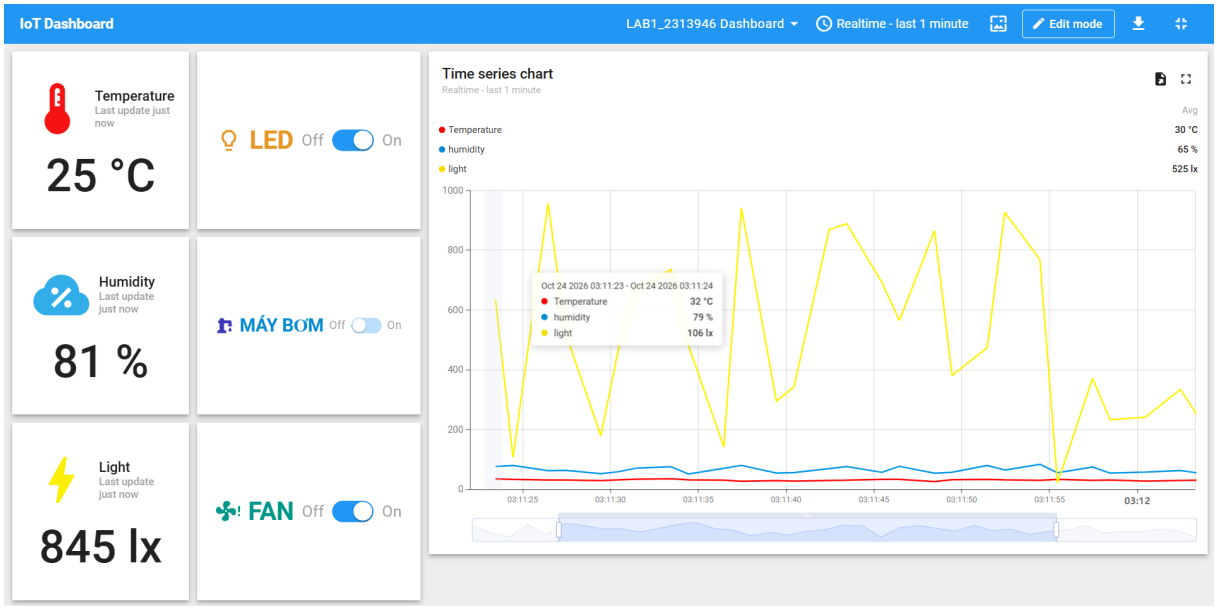
    # Xu ly setState: nhan lenh bat/tat tu Dashboard
    elif method.startswith("setState"):
        name = method[len("setState"):]
        if name in state:
            state[name] = bool(params)
            result = state[name]
            status_str = "ON" if result else "OFF"
            print(f"[DIEU KHIEN] {name} -> {status_str}")
            # Dong bo thuoc tinh len CoreIOT
            client.publish("v1/devices/me/attributes",
                           json.dumps({name.lower() + "_state": result}))

    # Bat buoc phan hoi ve topic response de Widget xac nhan lenh thanh cong
    client.publish(f"v1/devices/me/rpc/response/{request_id}",
                   json.dumps(result), qos=1)
```

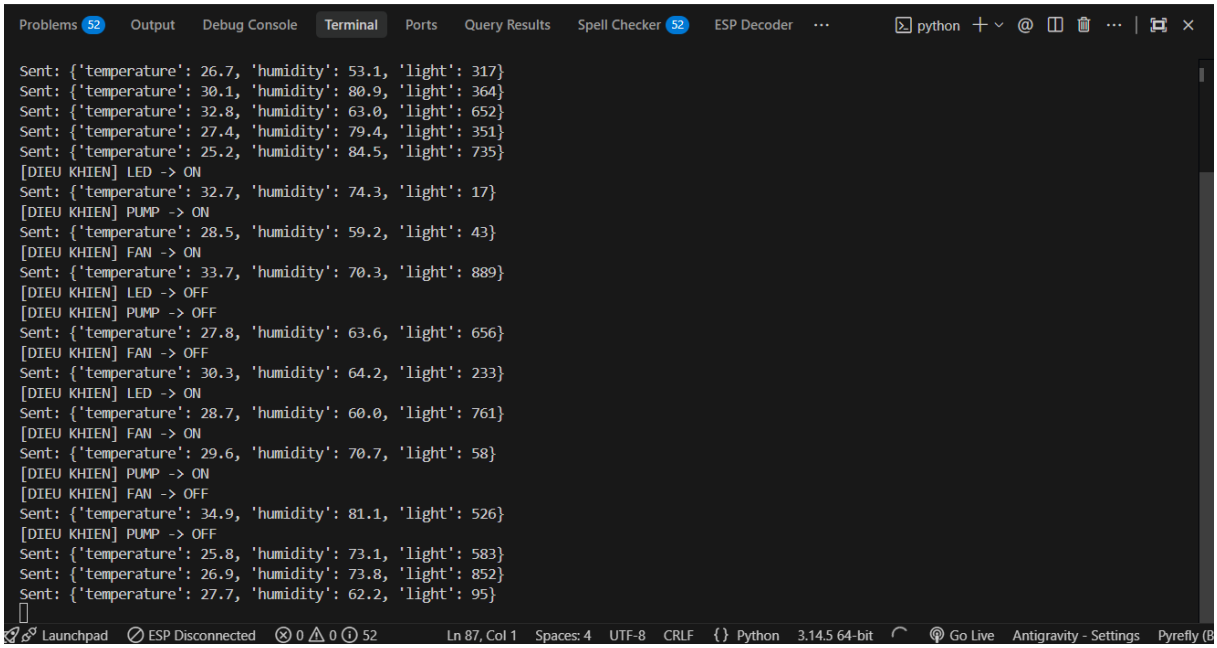
2.4.2 Hình ảnh minh chứng thực nghiệm



Hình 8: Cấu hình tab Behavior của widget Quạt (Fan) với đầy đủ 3 phương thức RPC



Hình 9: Dashboard hoàn chỉnh với 3 công tắc LED, MÁY BƠM, FAN sau khi đã điều khiển bật/tắt



Hình 10: Terminal hiển thị log nhận lệnh RPC và in rõ ràng các thông báo trạng thái `-> ON` / `-> OFF`

2.4.3 Trả lời câu hỏi bài tập 4

Câu hỏi 4: Dừng script và bật/tắt một công tắc. Widget hiển thị gì, và vì sao? Vì sao thiết bị phải gửi phản hồi tới v1/devices/me/rpc/response/{id}?

Trả lời:

- **Khi dừng script và gạt công tắc:** Widget công tắc sẽ chuyển sang biểu tượng quay vòng chờ đợi, sau vài giây sẽ báo lỗi tam giác đỏ kèm thông báo lỗi *"Failed to perform action: Request timeout"* và tự động nhảy ngược về trạng thái cũ ban đầu. Lý do là vì script Python đã ngắt kết nối nên không nhận được bản tin RPC từ server và không có tiến trình nào gửi gói tin phản hồi về máy chủ trong khoảng thời gian chờ (timeout).
- **Vì sao phải phản hồi về topic response:** Đây là nguyên lý của cơ chế gọi thủ tục từ xa hai chiều (**Two-way RPC**). Khi người dùng nhấn nút trên Dashboard, máy chủ CoreIOT gửi gói tin điều khiển kèm một định danh duy nhất `{id}` và treo tiến trình đợi. Thiết bị phần cứng bắt buộc phải gửi kết quả trả lời về đúng topic `v1/devices/me/rpc/response/{id}` để xác nhận lệnh đã được phần cứng tiếp nhận và thực thi thành công, từ đó widget trên giao diện mới chính thức cập nhật trạng thái hiển thị.

2.5 Bài 5: Chia sẻ Dashboard công khai (1.5 Điểm)

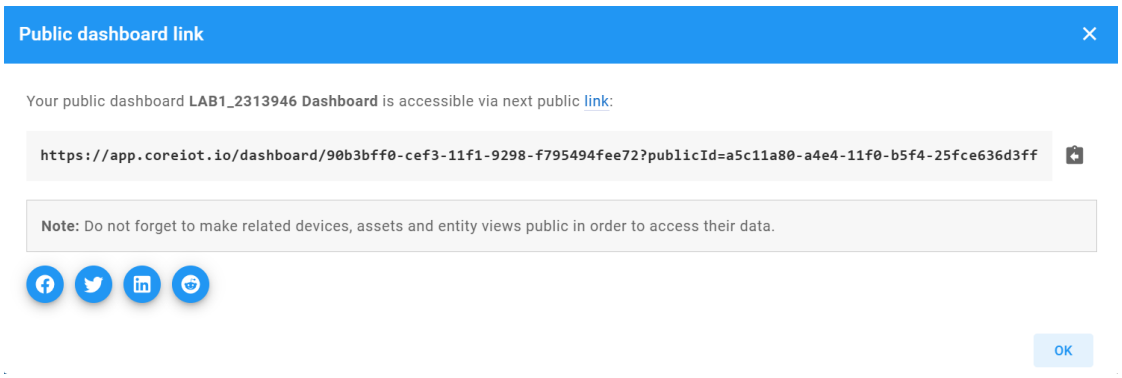
Yêu cầu đề bài (Trích phiếu bài tập)

- **Đặt nhóm dashboard chứa dashboard của bạn ở chế độ công khai (Make Public) và sao chép link công khai.**
- **Đặt nhóm thiết bị `LAB1_2313946_group` ở chế độ công khai.**
- **Mở liên kết trong cửa sổ trình duyệt ẩn danh / riêng tư (Incognito) để kiểm tra dữ liệu trực tiếp mà không cần đăng nhập.**
- **Cung cấp đường dẫn Public Dashboard và 2 ảnh minh chứng.**
- **Trả lời Câu hỏi 5.**

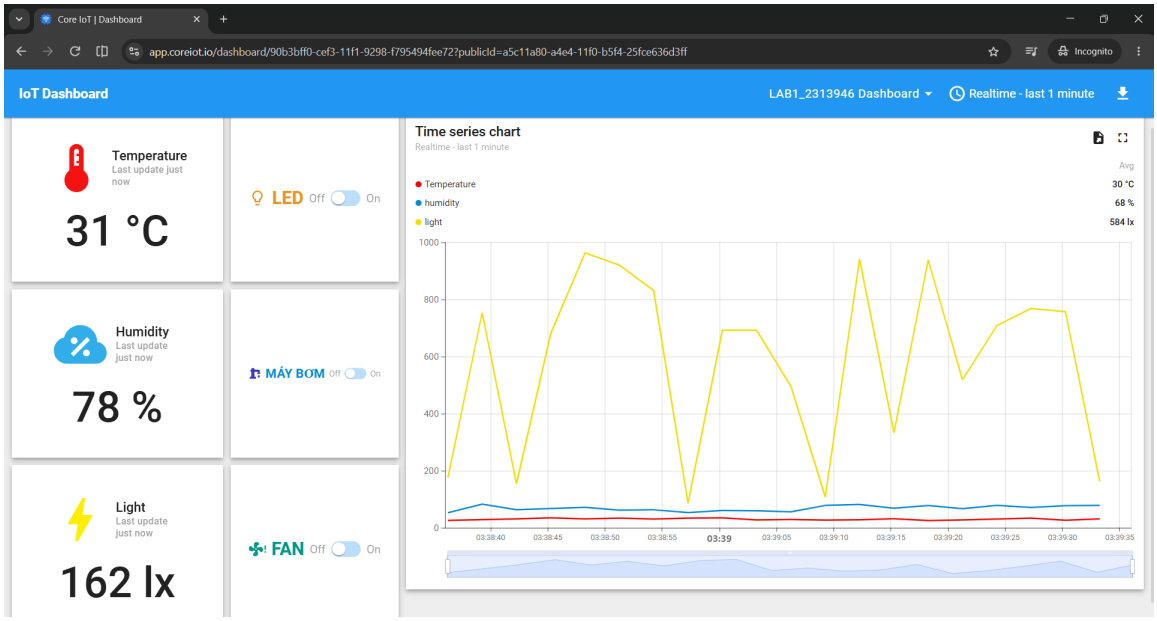
2.5.1 Kết quả thực nghiệm & Liên kết truy cập công khai

- **Đường dẫn Dashboard công khai (Public Link):**

https://app.coreiot.io/dashboard/90b3bff0-cef3-11f1-9298-f795494fee72?publicId=a5c11a80-a4e4-11f0-b5f4-25fce636d3ff



Hình 11: Hộp thoại Public dashboard link hiển thị đường link chia sẻ công khai



Hình 12: Toàn màn hình Dashboard mở trên trình duyệt ẩn danh (Incognito) hiển thị dữ liệu trực tiếp

2.5.2 Trả lời câu hỏi bài tập 5

Câu hỏi 5: Vì sao nhóm thiết bị cũng phải ở chế độ công khai? Điều gì có thể xảy ra nếu bạn đặt mọi dashboard và thiết bị trong tài khoản của mình ở chế độ công khai?

Trả lời:

- **Vì sao nhóm thiết bị cũng phải công khai:** Trên nền tảng CoreIOT/ThingsBoard, Dashboard chỉ là tầng giao diện hiển thị dữ liệu trực quan, trong khi quyền đọc/ghi dữ liệu viễn trắc thuộc quyền sở hữu của thực thể (Device Entity). Khi người dùng ẩn danh truy cập qua Public Link, nếu nhóm thiết bị vẫn ở chế độ riêng tư (Private) thì máy chủ sẽ từ chối cung cấp dữ liệu cảm biến cho phiên truy cập ẩn danh đó, khiến toàn bộ các thẻ giá trị và biểu đồ đều bị trắng dữ liệu (*"No data"*).
- **Hậu quả nếu đặt mọi thứ ở chế độ công khai:**
  1. *Rò rỉ dữ liệu nhạy cảm:* Toàn bộ số liệu đo đạc nội bộ, quy trình sinh hoạt, định vị vị trí và nhật ký vận hành đều bị công khai cho bất kỳ ai trên không gian mạng.
  2. *Mất an toàn điều khiển phần cứng:* Nếu trên các Dashboard công khai có chứa các widget điều khiển RPC (như công tắc Bơm, Đèn, Quạt, Relay), bất kỳ người lạ nào cũng có thể tự do bật/tắt thiết bị vật lý thật ngoài đời thực, dẫn đến nguy cơ xung đột vận hành, hỏng hóc máy móc hoặc gây ra các sự cố cháy nổ nguy hiểm.

Tài liệu

[1] CoreIOT Documentation. (2024). *CoreIOT / ThingsBoard Platform Architecture and APIs*.

[2] Espressif Systems. (2023). *ESP32-S3 Series SoC Technical Reference Manual*.

[3] Real Time Engineers Ltd. (2024). *FreeRTOS Kernel Developer Guide*.

[4] Khoa KH&KT Máy tính - ĐH Bách Khoa ĐHQG-HCM. (2026). *Tài liệu hướng dẫn thực hành IoT trên YoloUNO (ESP32-S3)*.